Simple Definition

CodeIgniter is a tool that helps developers write clean**,** organized PHP code using a structured approach called MVC (Model–View–Controller).

How It Works (MVC Concept)

- **Model** → Handles database (data logic)
- **View** → Handles UI (HTML design)
- **Controller** → Connects Model and View

This structure makes your project easy to manage and scalable

Key Features

Lightweight and fast
Easy to learn (great for beginners)
No heavy configuration required
Built-in libraries (database, session, validation, etc.)
Secure (helps prevent common web attacks)

Example Use

You can use CodeIgniter to build:

- Student management system
- Login & registration system
- Blog website
- E-commerce site

Why Use CodeIgniter?

Compared to plain PHP:

- Less code required
- Better organization
- Faster development

Simple Example

Without framework:

```
echo "Hello World";
```

With CodeIgniter (Controller):

```
public function index()
{
    return "Hello CodeIgniter";
}
```

In Short

CodeIgniter = PHP + Structure + Speed + Simplicity

Application Structure

To get the most out of CodeIgniter, you need to understand how the application is structured, by default, and what you can change to meet the needs of your application.

- Default Directories
  - app
  - system
  - public
  - writable
  - tests
- Modifying Directory Locations

Default Directories

A fresh install has five directories:

- **app**
- **public**
- **writable**
- **tests**
- **vendor** or **system**

Each of these directories has a very specific part to play.

app

The **app** directory is where all of your application code lives. This comes with a default directory structure that works well for many applications.

The following folders make up the basic contents:

```
app/
    Config/         Stores the configuration files
    Controllers/    Controllers determine the program flow
    Database/       Stores the database migrations and seeds files
    Filters/        Stores filter classes that can run before and after controller
    Helpers/        Helpers store collections of standalone functions
    Language/       Multiple language support reads the language strings from here
    Libraries/      Useful classes that don't fit in another category
    Models/         Models work with the database to represent the business entities
    ThirdParty/     ThirdParty libraries that can be used in application
    Views/          Views make up the HTML that is displayed to the client
```

Because the app directory is already namespaced, you should feel free to modify the structure of this directory to suit your application's needs.

For example, you might decide to start using the Repository pattern and Entities to work with your data. In this case, you could rename the Models directory to Repositories, and add a new Entities directory.

All files in this directory live under the App namespace, though you are free to change that in **app/Config/Constants.php**.

system

Note :

If you install CodeIgniter with Composer, the system is located in vendor/codeigniter4/framework/system.

This directory stores the files that make up the framework, itself. While you have a lot of flexibility in how you use the application directory, the files in the system directory should never be modified. Instead, you should extend the classes, or create new classes, to provide the desired functionality.

All files in this directory live under the CodeIgniter namespace.

public

The public folder holds the browser-accessible portion of your web application, preventing direct access to your source code.

It contains the main .htaccess file, index.php, and any application assets that you add, like CSS, javascript, or images.

This folder is meant to be the “web root” of your site, and your web server would be configured to point to it.

writable

This directory holds any directories that might need to be written to in the course of an application’s life. This includes directories for storing cache files, logs, and any uploads a user might send.

You should add any other directories that your application will need to write to here. This allows you to keep your other primary directories non-writable as an added security measure.

tests

This directory is set up to hold your test files. The **_support** directory holds various mock classes and other utilities that you can use while writing your tests.

This directory does not need to be transferred to your production servers.

Modifying Directory Locations

If you’ve relocated any of the main directories, you can change the configuration settings inside **app/Config/Paths.php**.

Please read Managing your Applications.

Models, Views, and Controllers

What is MVC?

Whenever you create an application, you have to find a way to organize the code to make it simple to locate the proper files and make it simple to maintain. Like most of the web frameworks, CodeIgniter uses the Model, View, Controller (MVC) pattern to organize the files. This keeps the data, the presentation, and flow through the application as separate parts.

It should be noted that there are many views on the exact roles of each element, but this document describes our take on it. If you think of it differently, you're free to modify how you use each piece as you need.

Models manage the data of the application and help to enforce any special business rules the application might need.

Views are simple files, with little to no logic, that display the information to the user.

Controllers act as glue code, marshaling data back and forth between the view (or the user that's seeing it) and the data storage.

At their most basic, controllers and models are simply classes that have a specific job. They are not the only class types that you can use, obviously, but they make up the core of how this framework is designed to be used. They even have designated directories in the **app** directory for their storage, though you're free to store them wherever you desire, as long as they are properly namespaced. We will discuss that in more detail below.

Let's take a closer look at each of these three main components.

The Components

Views

Views are the simplest files and are typically HTML with very small amounts of PHP. The PHP should be very simple, usually just displaying a variable's contents, or looping over some items and displaying their information in a table.

Views get the data to display from the controllers, who pass it to the views as variables that can be displayed with simple `echo` calls. You can also display other views within a view, making it pretty simple to display a common header or footer on every page.

Views are generally stored in **app/Views**, but can quickly become unwieldy if not organized in some fashion. CodeIgniter does not enforce any type of organization, but a good rule of thumb would be to create a new directory in the **Views** directory for each controller. Then, name views by the method name. This makes them very easy to find later on. For example, a user's profile might be displayed in a controller named `User`, and a method named `profile`. You might store the view file for this method in **app/Views/user/profile.php**.

That type of organization works great as a base habit to get into. At times you might need to organize it differently. That's not a problem. As long as CodeIgniter can find the file, it can display it.

Models

A model's job is to maintain a single type of data for the application. This might be users, blog posts, transactions, etc. In this case, the model's job has two parts: enforce business rules on the data as it is pulled from, or put into, the database; and handle the actual saving and retrieval of the data from the database.

For many developers, the confusion comes in when determining what business rules are enforced. It simply means that any restrictions or requirements on the data is handled by the model. This might include normalizing raw data before it's saved to meet company standards, or formatting a column in a certain way before handing it to the controller. By keeping these business requirements in the model, you won't repeat code throughout several controllers and accidentally miss updating an area.

Models are typically stored in **app/Models**, though they can use a namespace to be grouped however you need.

Controllers

Controllers have a couple of different roles to play. The most obvious one is that they receive input from the user and then determine what to do with it. This often involves passing the data to a model to save it, or requesting data from the model that is then passed on to the view to be displayed. This also includes loading up other utility classes, if needed, to handle specialized tasks that is outside of the purview of the model.

The other responsibility of the controller is to handle everything that pertains to HTTP requests - redirects, authentication, web safety, encoding, etc. In short, the controller is where you make sure that people are allowed to be there, and they get the data they need in a format they can use.

Controllers are typically stored in **app/Controllers**, though they can use a namespace to be grouped however you need.

What is a Controller?

A Controller is simply a class file that handles an HTTP request. URI Routing associates a URI with a controller. It returns a view string or `Response` object.

Every controller you create should extend `BaseController` class. This class provides several features that are available to all of your controllers.

Installation Steps for CodeIgniter

Download CodeIgniter

- Go to official site:
  ☞ https://codeigniter.com/download
- Download **CodeIgniter 4 (ZIP file)**

Extract Files

- Extract ZIP file

- Rename folder to:

```
ci-app
```

Move to XAMPP htdocs

Copy folder to:

```
C:\xampp\htdocs\ci-app
```

Start XAMPP

Open XAMPP Control Panel:

- Start **Apache**
- Start **MySQL** (optional)

Configure Base URL (Important)

Open file:

```
app/Config/App.php
```

Find:

```
public $baseURL = 'http://localhost:8080/';
```

Change to:

```
public $baseURL = 'http://localhost/ci4/public/';
```

Open Project in Browser

Open:

```
http://localhost/ci4/public/
```

You should see ***CodeIgniter welcome page***

Menual Installation using Composer

Use this if Composer is working on your system

Open Command Prompt

```
cd C:\xampp\htdocs
```

Run Command

```
composer create-project codeigniter4/appstarter ci-app
```

Start Development Server

```
cd ci4
php spark serve
```

Open Browser

```
http://localhost:8080
```

Common Errors & Fixes

1. “Page Not Found”

Check URL:

```
http://localhost/ci4/public/
```

2. mod_rewrite not enabled

Enable in XAMPP:

- Open `httpd.conf`
- Remove `#` from:

```
LoadModule rewrite_module modules/mod_rewrite.so
```

3. Permission Error

Run XAMPP as Administrator

4. Composer not working

Check version:

```
composer -V
```